Data 236- Distributed Systems

Homework-8

Kruthika Virupakshappa

Part 1-Backend Service

1. **Docker Containers Running** - Command: docker ps

```yaml
services:
 zookeeper:
   image: confluentinc/cp-zookeeper:7.4.0
   container_name: zookeeper
   environment:
     ZOOKEEPER_CLIENT_PORT: 2181
     ZOOKEEPER_TICK_TIME: 2000
   ports:
    - "2181:2181"


 kafka:
   image: confluentinc/cp-kafka:7.4.0
   container_name: kafka
   depends_on:
    - zookeeper
   ports:
    - "9092:9092"
   environment:
     KAFKA_BROKER_ID: 1
     KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
     KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
     KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
     KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"
```

```
(env) Kruthika@Kruthikas-MacBook-Air Part1 % docker ps
CONTAINER ID   IMAGE                             COMMAND                  CREATED       STATUS       PORTS                                                NAMES
979fdaca6955   confluentinc/cp-kafka:7.4.0       "/etc/confluent/dock…"   3 hours ago   Up 3 hours   0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp          kafka
adfc4669053d   confluentinc/cp-zookeeper:7.4.0   "/etc/confluent/dock…"   3 hours ago   Up 3 hours   0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp          zookeeper
(env) Kruthika@Kruthikas-MacBook-Air Part1 %
```

part1	-	-	-	1.43%	551.7MB / 15.31(	7.04%	0B / 12.67MB	5.52MB / 3.79M	
zookeeper	adfc4669053d	confluentin	2181:2181	0.22%	121.8MB / 7.65GI	1.55%	0B / 2.57MB	177KB / 121KB	
kafka	979fdaca6955	confluentin	9092:9092	1.21%	429.9MB / 7.65GI	5.49%	0B / 10.1MB	5.35MB / 3.67M	

○ Worker (consumer) service started (shows: "Consumer is ready and listening…")

```
2026-04-02 20:30:45,136 [worker] <BrokerConnection client_id=kafka-python-2.3.0, node_id=bootstrap-0 host=localhost:9092 <connected> [IPv6 (':
:1', 9092, 0, 0)]>: Connection complete.
2026-04-02 20:30:45,136 [worker] Updating subscribed topics to: ('user_requests',)
2026-04-02 20:30:45,136 [worker] Consumer is ready and listening on topic: user_requests
2026-04-02 20:30:45,166 [worker] Topic user_requests is not available during auto-create initialization
2026-04-02 20:30:45,167 [worker] <BrokerConnection client_id=kafka-python-2.3.0, node_id=1 host=localhost:9092 <connecting> [IPv6 ('::1', 9092
, 0, 0)]>: connecting to localhost:9092 [('::1', 9092, 0, 0) IPv6]
2026-04-02 20:30:45,269 [worker] <BrokerConnection client_id=kafka-python-2.3.0, node_id=1 host=localhost:9092 <connected> [IPv6 ('::1', 9092,
 0, 0)]>: Connection complete.
2026-04-02 20:30:45,269 [worker] <BrokerConnection client_id=kafka-python-2.3.0, node_id=bootstrap-0 host=localhost:9092 <connected> [IPv6 (':
:1', 9092, 0, 0)]>: Closing connection.
2026-04-02 20:30:45,709 [worker] Coordinator for group/user-worker-group is BrokerMetadata(nodeId='coordinator-1', host='localhost', port=9092
, rack=None)
2026-04-02 20:30:45,709 [worker] Discovered coordinator coordinator-1 for group user-worker-group
2026-04-02 20:30:45,709 [worker] Starting new heartbeat thread
2026-04-02 20:30:45,710 [worker] Revoking previously assigned partitions set() for group user-worker-group
2026-04-02 20:30:45,710 [worker] Failed to join group user-worker-group: NodeNotReadyError: coordinator-1
2026-04-02 20:30:45,711 [worker] <BrokerConnection client_id=kafka-python-2.3.0, node_id=coordinator-1 host=localhost:9092 <connecting> [IPv6
('::1', 9092, 0, 0)]>: connecting to localhost:9092 [('::1', 9092, 0, 0) IPv6]
2026-04-02 20:30:45,812 [worker] Failed to join group user-worker-group: NodeNotReadyError: coordinator-1
2026-04-02 20:30:45,813 [worker] <BrokerConnection client_id=kafka-python-2.3.0, node_id=coordinator-1 host=localhost:9092 <connected> [IPv6 (
'::1', 9092, 0, 0)]>: Connection complete.
2026-04-02 20:30:45,918 [worker] (Re-)joining group user-worker-group
2026-04-02 20:30:45,946 [worker] Received member id kafka-python-2.3.0-dbe3cb27-0231-4b77-908f-764c015cc1f1 for group user-worker-group; will
retry join-group
```

○ FastAPI service running (shows server started on http://127.0.0.1:8000)

```python
@asynccontextmanager
async def lifespan(app: FastAPI):
    global producer
    producer = KafkaProducer(
        bootstrap_servers=BOOTSTRAP_SERVERS,
        value_serializer=lambda v: json.dumps(v).encode("utf-8"),
    )
    listener_thread = threading.Thread(target=response_listener, daemon=True)
    listener_thread.start()
    logger.info("FastAPI service started on http://127.0.0.1:8000")
    yield
    _shutdown_event.set()
    producer.close()



app = FastAPI(lifespan=lifespan)
```

```
PROBLEMS   OUTPUT   TERMINAL   PORTS   SPELL CHECKER

(env) Kruthika@Kruthikas-MacBook-Air Part1 % uvicorn main:app --reload
INFO:     Will watch for changes in these directories: ['/Users/Kruthika/Documents/Data 236 Distributed Sys/Homework/HW8/Part1']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [19073] using StatReload
INFO:     Started server process [20150]
INFO:     Waiting for application startup.
2026-04-02 20:36:06,030 [fastapi] <BrokerConnection client_id=kafka-python-producer-1, node_id=bootstrap-0 host=localhost:9092 <connecting> [I
Pv6 ('::1', 9092, 0, 0)]>: connecting to localhost:9092 [('::1', 9092, 0, 0) IPv6]
2026-04-02 20:36:06,041 [fastapi] <BrokerConnection client_id=kafka-python-producer-1, node_id=bootstrap-0 host=localhost:9092 <checking_api_v
ersions_recv> [IPv6 ('::1', 9092, 0, 0)]>: Broker version identified as 2.6
2026-04-02 20:36:06,041 [fastapi] <BrokerConnection client_id=kafka-python-producer-1, node_id=bootstrap-0 host=localhost:9092 <connected> [IP
v6 ('::1', 9092, 0, 0)]>: Connection complete.
2026-04-02 20:36:06,042 [fastapi] FastAPI service started on http://127.0.0.1:8000
INFO:     Application startup complete.
2026-04-02 20:36:06,043 [fastapi] <BrokerConnection client_id=kafka-python-2.3.0, node_id=bootstrap-0 host=localhost:9092 <connecting> [IPv6 (
'::1', 9092, 0, 0)]>: connecting to localhost:9092 [('::1', 9092, 0, 0) IPv6]
2026-04-02 20:36:06,049 [fastapi] <BrokerConnection client_id=kafka-python-2.3.0, node_id=bootstrap-0 host=localhost:9092 <checking_api_versio
ns_recv> [IPv6 ('::1', 9092, 0, 0)]>: Broker version identified as 2.6
2026-04-02 20:36:06,049 [fastapi] <BrokerConnection client_id=kafka-python-2.3.0, node_id=bootstrap-0 host=localhost:9092 <connected> [IPv6 ('
::1', 9092, 0, 0)]>: Connection complete.
```

2. **Execute CREATE_USER and GET_USER operations** (from homework_demo.py or API call)

```python
def demo_create_user():
    print("\n=== CREATE_USER ===")
    payload = {
        "name": "Alice Johnson",
        "email": "alice@example.com",
        "age": 30,
    }
    print(f"Request:  POST /users  {payload}")
    response = requests.post(f"{BASE_URL}/users", json=payload)
    data = response.json()
    print(f"Response: {data}")
    assert response.status_code == 200, f"Expected 200, got {response.status_code}"
    assert data["success"] is True
    return data["userId"]
```

```python
def demo_get_user(user_id: str):
    print("\n=== GET_USER ===")
    print(f"Request:  GET /users/{user_id}")
    response = requests.get(f"{BASE_URL}/users/{user_id}")
    data = response.json()
    print(f"Response: {data}")
    assert response.status_code == 200, f"Expected 200, got {response.status_code}"
    assert data["success"] is True
    assert data["user"]["userId"] == user_id
```

```python
def demo_get_nonexistent_user():
    print("\n=== GET_USER (non-existent) ===")
    bad_id = "00000000-0000-0000-0000-000000000000"
    print(f"Request:  GET /users/{bad_id}")
    response = requests.get(f"{BASE_URL}/users/{bad_id}")
    data = response.json()
    print(f"Response (status {response.status_code}): {data}")
    assert response.status_code == 404
```

```
(env) Kruthika@Kruthikas-MacBook-Air Part1 % python homework_demo.py

=== CREATE_USER ===
Request:  POST /users  {'name': 'Alice Johnson', 'email': 'alice@example.com', 'age': 30}
Response: {'success': True, 'userId': 'a4e7f09e-5a00-4dd8-8893-7cd0e147a4a9', 'message': 'User created'}

=== GET_USER ===
Request:  GET /users/a4e7f09e-5a00-4dd8-8893-7cd0e147a4a9
Response: {'success': True, 'user': {'userId': 'a4e7f09e-5a00-4dd8-8893-7cd0e147a4a9', 'name': 'Alice Johnson', 'email': 'alice@example.com
', 'age': 30.0}}

=== GET_USER (non-existent) ===
Request:  GET /users/00000000-0000-0000-0000-000000000000
Response (status 404): {'detail': 'User 00000000-0000-0000-0000-000000000000 not found'}

All demo operations completed successfully.
(env) Kruthika@Kruthikas-MacBook-Air Part1 %
```

3. **Show logs of:**

```python
def send_and_wait(data: dict) -> dict:
    correlation_id = str(uuid.uuid4())
    event = threading.Event()
    _pending[correlation_id] = event

    message = {
        "correlation_id": correlation_id,
        "reply_to": RESPONSE_TOPIC,
        "data": data,
    }
    logger.info(
        "Sending message to user_requests: correlation_id=%s operation=%s",
        correlation_id,
        data.get("operation"),
    )
    producer.send(REQUEST_TOPIC, value=message)
    producer.flush()

    if not event.wait(timeout=RESPONSE_TIMEOUT):
        _pending.pop(correlation_id, None)
        raise HTTPException(status_code=504, detail="Worker response timed out")

    _pending.pop(correlation_id, None)
    return _results.pop(correlation_id, {})
```

- message sent to user_requests topic

```python
@app.post("/users")
def create_user(req: CreateUserRequest):
    result = send_and_wait({
        "operation": "CREATE_USER",
        "name": req.name,
        "email": req.email,
        "age": req.age,
    })
    if not result.get("success"):
        raise HTTPException(status_code=400, detail=result.get("message"))
    return result


@app.get("/users/{user_id}")
def get_user(user_id: str):
```

```
    result = send_and_wait({
        "operation": "GET_USER",
        "userId": user_id,
    })
    if not result.get("success"):
        raise HTTPException(status_code=404, detail=result.get("message"))
    return result
```

2026-04-02 20:41:40,163 [fastapi] Sending message to user_requests: correlation_id=86e5527a-e761-462a-873f-92a63fec4cee operation=CREATE_USER

2026-04-02 20:41:40,200 [fastapi] Sending message to user_requests: correlation_id=7bb5a631-7424-4327-a785-418b9b8c87bd operation=GET_USER

2026-04-02 20:41:40,209 [fastapi] Sending message to user_requests: correlation_id=36e4c339-3c7a-4ba6-b222-08bceca0c860 operation=GET_USER

- message received and processed by worker

```
    for message in consumer:
        try:
            msg = message.value
            logger.info("Message received from user_requests: %s", json.dumps(msg))

            response, reply_to = process_message(msg)

            producer.send(reply_to, value=response)
            producer.flush()
            logger.info(
                "Response sent to %s: %s", reply_to, json.dumps(response)
            )
        except Exception as exc:
            logger.error("Error processing message: %s", exc)
```

2026-04-02 20:41:40,188 [worker] Processing message: correlation_id=86e5527a-e761-462a-873f-92a63fec4cee operation=CREATE_USER
2026-04-02 20:41:40,188 [worker] Created user: userId=a4e7f09e-5a00-4dd8-8893-7cd0e147a4a9 name=Alice Johnson

2026-04-02 20:41:40,203 [worker] Message received from user_requests: {"correlation_id": "7bb5a631-7424-4327-a785-418b9b8c87bd", "reply_to": "user_responses", "data": {"operation": "GET_USER", "userId": "a4e7f09e-5a00-4dd8-8893-7cd0e147a4a9"}}
2026-04-02 20:41:40,204 [worker] Processing message: correlation_id=7bb5a631-7424-4327-a785-418b9b8c87bd operation=GET_USER
2026-04-02 20:41:40,204 [worker] Retrieved user: userId=a4e7f09e-5a00-4dd8-8893-7cd0e147a4a9

2026-04-02 20:41:40,212 [worker] Message received from user_requests: {"correlation_id": "36e4c339-3c7a-4ba6-b222-08bceca0c860", "reply_to": "user_responses", "data": {"operation": "GET_USER", "userId": "00000000-0000-0000-0000-000000000000"}}

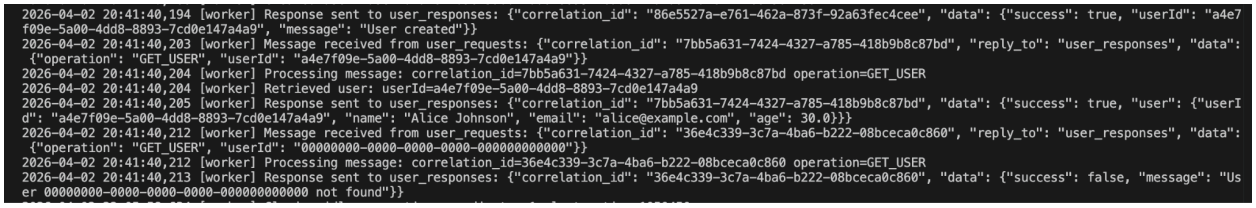
2026-04-02 20:41:40,212 [worker] Processing message: correlation_id=36e4c339-3c7a-4ba6-b222-08bceca0c860 operation=GET_USER

- response sent to user_responses topic

```
    for message in consumer:
        try:
            msg = message.value
            logger.info("Message received from user_requests: %s", json.dumps(msg))
            response, reply_to = process_message(msg)
            producer.send(reply_to, value=response)
            producer.flush()
            logger.info(
                "Response sent to %s: %s", reply_to, json.dumps(response)
            )
        except Exception as exc:
            logger.error("Error processing message: %s", exc)
```

2026-04-02 20:41:40,194 [worker] Response sent to user_responses: {"correlation_id": "86e5527a-e761-462a-873f-92a63fec4cee", "data": {"success": true, "userId": "a4e7f09e-5a00-4dd8-8893-7cd0e147a4a9", "message": "User created"}}
2026-04-02 20:41:40,205 [worker] Response sent to user_responses: {"correlation_id": "7bb5a631-7424-4327-a785-418b9b8c87bd", "data": {"success": true, "user": {"userId": "a4e7f09e-5a00-4dd8-8893-7cd0e147a4a9", "name": "Alice Johnson", "email": "alice@example.com", "age": 30.0}}}

2026-04-02 20:41:40,213 [worker] Response sent to user_responses: {"correlation_id": "36e4c339-3c7a-4ba6-b222-08bceca0c860", "data": {"success": false, "message": "User 00000000-0000-0000-0000-000000000000 not found"}}



- response received by FastAPI and returned to client

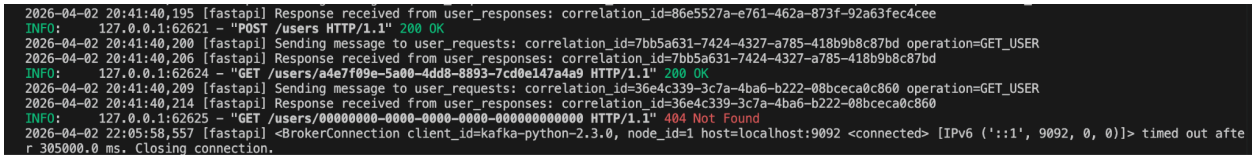
```
def response_listener():
    """Background thread: consume from user_responses and resolve pending requests."""
    consumer = KafkaConsumer(
        RESPONSE_TOPIC,
        bootstrap_servers=BOOTSTRAP_SERVERS,
        auto_offset_reset="latest",
        enable_auto_commit=True,
        group_id="fastapi-response-group",
        value_deserializer=lambda m: json.loads(m.decode("utf-8")),
```

```
        consumer_timeout_ms=1000,
    )
    logger.info("Response listener started on topic: %s", RESPONSE_TOPIC)
    while not _shutdown_event.is_set():
        try:
            for message in consumer:
                msg = message.value
                cid = msg.get("correlation_id")
                logger.info(
                    "Response received from user_responses: correlation_id=%s", cid
                )
                if cid and cid in _pending:
                    _results[cid] = msg.get("data", {})
                    _pending[cid].set()
        except Exception:
            pass
    consumer.close()
```

2026-04-02 20:41:40,195 [fastapi] Response received from user_responses: correlation_id=86e5527a-e761-462a-873f-92a63fec4cee
INFO: 127.0.0.1:62621 - "POST /users HTTP/1.1" 200 OK

2026-04-02 20:41:40,206 [fastapi] Response received from user_responses: correlation_id=7bb5a631-7424-4327-a785-418b9b8c87bd
INFO: 127.0.0.1:62624 - "GET /users/a4e7f09e-5a00-4dd8-8893-7cd0e147a4a9 HTTP/1.1" 200 OK

2026-04-02 20:41:40,214 [fastapi] Response received from user_responses: correlation_id=36e4c339-3c7a-4ba6-b222-08bceca0c860
INFO: 127.0.0.1:62625 - "GET /users/00000000-0000-0000-0000-000000000000 HTTP/1.1" 404 Not Found



Part 2-3-Agent Mini-System with Kafka + Langchain

Build a tiny system with 3 agents that communicate through Kafka:
1. **Planner**: Makes a plan for the question.

```
SYSTEM_PROMPT = (
   "You are a Planner agent. Given a question, produce a clear numbered "
   "step-by-step TODO plan (3-5 steps) that a writer can follow to answer it. "
   "Be concise. Output only the numbered list, nothing else."
)
```

```
def make_plan(llm: ChatOllama, question: str) -> str:
    messages = [
        SystemMessage(content=SYSTEM_PROMPT),
        HumanMessage(content=f"Question: {question}"),
    ]
    return llm.invoke(messages).content.strip()
```

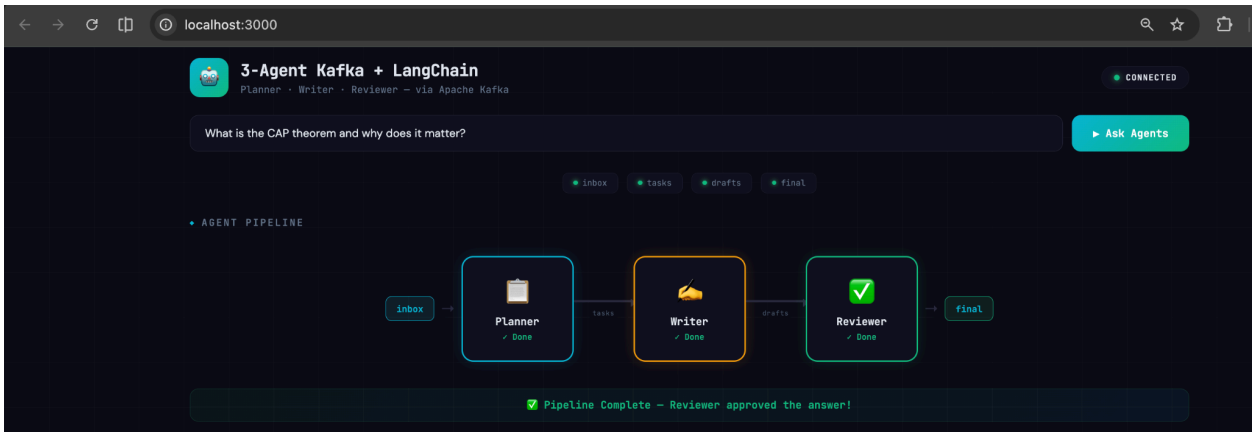
2. **Writer**: Writes a short answer using LangChain.

```
SYSTEM_PROMPT = (
    "You are a Writer agent. Given a question and a step-by-step plan, write a "
    "clear, concise answer (3-5 sentences) that directly addresses the question "
    "by following the plan. Use plain language. Output only the answer text."
)


def write_answer(llm: ChatOllama, question: str, plan: str) -> str:
    user_content = (
        f"Question: {question}\n\n"
        f"Plan to follow:\n{plan}\n\n"
        "Write the answer now."
    )
    messages = [
        SystemMessage(content=SYSTEM_PROMPT),
        HumanMessage(content=user_content),
    ]
    return llm.invoke(messages).content.strip()
```

3. **Reviewer**: Checks the answer and approves it.

```
SYSTEM_PROMPT = (
    "You are a Reviewer agent. Evaluate the given answer for accuracy and clarity. "
    "Respond with ONLY a valid JSON object -- no markdown, no code fences, no extra text:\n"
    '{"status": "approved", "answer": "<the answer verbatim>", "feedback": "<brief
feedback>"}'
)
def review_answer(llm: ChatOllama, question: str, answer: str) -> dict:
    user_content = (
        f"Question: {question}\n\n"
        f"Answer to review:\n{answer}"
    )
    messages = [
        SystemMessage(content=SYSTEM_PROMPT),
        HumanMessage(content=user_content),
    ]
    raw = llm.invoke(messages).content.strip()

    try:
        result = json.loads(raw)
    except json.JSONDecodeError:
```

```
        result = {
            "status": "approved",
            "answer": answer,
            "feedback": raw,
        }
    return result
```



● Screenshot or log lines showing each agent received and sent messages.

```
async def agent_think(agent_id: str, data: dict) -> str:
    """Build the prompt per agent and call LangChain asynchronously."""
    agent = AGENTS[agent_id]
    question = data.get("question", "")
    plan = data.get("plan", "")
    answer = data.get("answer", "")
    print(f"[{agent['name']}] Calling OpenAI... question={question[:60]}")

    if agent_id == "planner":
        user_content = f"Question: {question}"
        temperature = 0.3
    elif agent_id == "writer":
        user_content = (
            f"Question: {question}\n\n"
            f"Plan to follow:\n{plan}\n\n"
            "Write the answer now."
        )
        temperature = 0.5
    else:  # reviewer
        user_content = (
            f"Question: {question}\n\n"
            f"Answer to review:\n{answer}"
        )
        temperature = 0.2

    llm = get_llm(temperature)
    messages = [
```

```
        SystemMessage(content=agent["system_prompt"]),
        HumanMessage(content=user_content),
    ]
    response = await asyncio.wait_for(llm.ainvoke(messages), timeout=60)
    print(f"[{agent['name']}] OpenAI responded.")
    return response.content.strip()
```

```
backend-1  | ✔ Agent [Planner] listening on 'inbox'
backend-1  | ✔ Agent [Writer] listening on 'tasks'
backend-1  | ✔ Agent [Reviewer] listening on 'drafts'
backend-1  | INFO:     172.20.0.5:57874 - "WebSocket /ws" [accepted]
backend-1  | INFO:     connection open
backend-1  | INFO:     172.20.0.5:57882 - "POST /api/task HTTP/1.1" 200 OK
backend-1  | [Planner] ← Message received on 'inbox'
backend-1  | [Planner] Calling OpenAI... question=What is the CAP theorem and why does it matter?
backend-1  | [Planner] OpenAI responded.
backend-1  | [Writer] ← Message received on 'tasks'
backend-1  | [Writer] Calling OpenAI... question=What is the CAP theorem and why does it matter?
backend-1  | [Writer] OpenAI responded.
backend-1  | [Reviewer] ← Message received on 'drafts'
backend-1  | [Reviewer] Calling OpenAI... question=What is the CAP theorem and why does it matter?
backend-1  | [Reviewer] OpenAI responded.
```

● Screenshot of the message in topic final with "status": "approved".

```
    try:
        async for msg in consumer:
            data = msg.value
            task_id = data.get("task_id", str(uuid.uuid4())[:8])
            print(f"[{agent['name']}] ← Message received on '{topic}'")

            # Broadcast: agent started
            await manager.broadcast({
                "type": "agent_start",
                "agent": agent_id,
                "task_id": task_id,
                "timestamp": datetime.utcnow().isoformat(),
            })

            # Broadcast: agent thinking
            await manager.broadcast({
                "type": "agent_thinking",
                "agent": agent_id,
                "task_id": task_id,
                "timestamp": datetime.utcnow().isoformat(),
            })

            # Do the work
            result = await agent_think(agent_id, data)

            # Broadcast: agent completed
            await manager.broadcast({
                "type": "agent_complete",
                "agent": agent_id,
                "task_id": task_id,
```

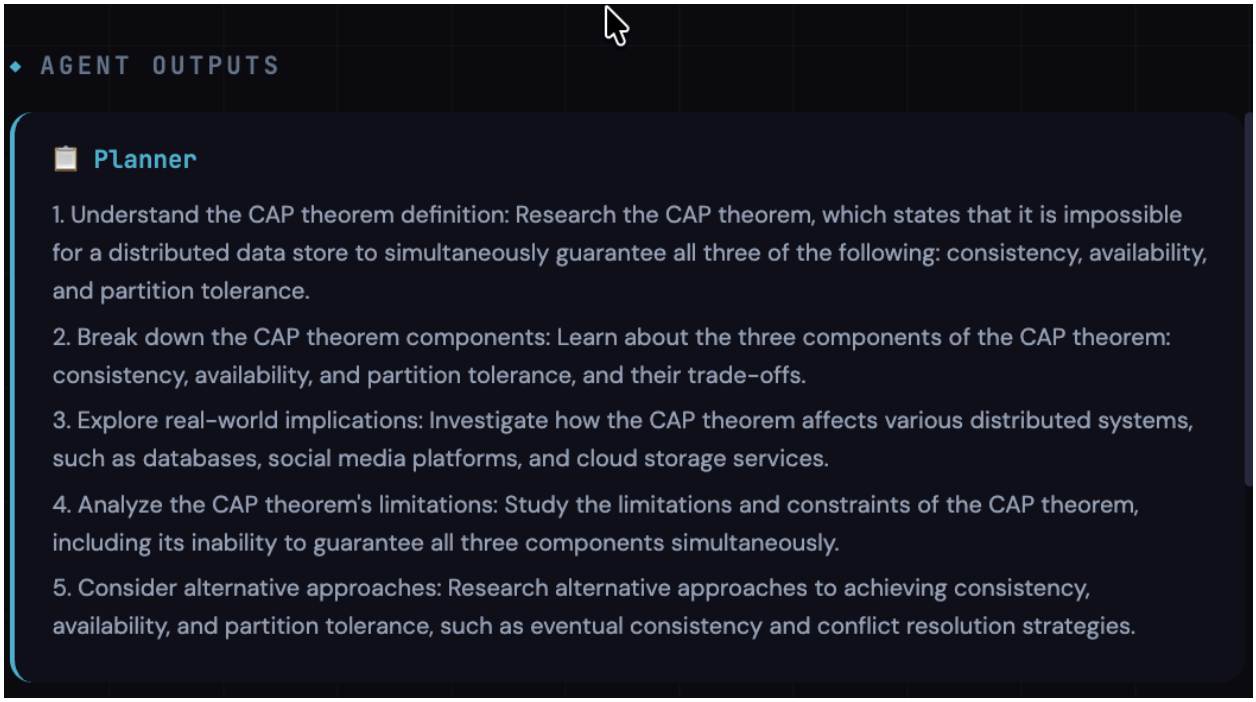
```
            "content": result,
            "timestamp": datetime.utcnow().isoformat(),
        })

        # Build the outgoing message for the next agent
        out_msg: dict = {"task_id": task_id, "from_agent": agent_id}

        if agent_id == "planner":
            out_msg["question"] = data.get("question", "")
            out_msg["plan"] = result
        elif agent_id == "writer":
            out_msg["question"] = data.get("question", "")
            out_msg["plan"] = data.get("plan", "")
            out_msg["answer"] = result
        else:  # reviewer -- parse JSON or wrap raw text
            try:
                parsed = json.loads(result)
                out_msg.update(parsed)
            except json.JSONDecodeError:
                out_msg["status"] = "approved"
                out_msg["answer"] = data.get("answer", result)
                out_msg["feedback"] = result

        # Broadcast: kafka message sent
        await manager.broadcast({
            "type": "kafka_message",
            "from_agent": agent_id,
            "to_topic": out_topic,
            "task_id": task_id,
            "timestamp": datetime.utcnow().isoformat(),
        })

        await publish(out_topic, out_msg)

        # Reviewer = pipeline complete
        if agent_id == "reviewer":
            await manager.broadcast({
                "type": "pipeline_complete",
                "task_id": task_id,
                "final_output": out_msg,
                "timestamp": datetime.utcnow().isoformat(),
            })
    finally:
        await consumer.stop()
```

```
(env) Kruthika@Kruthikas-MacBook-Air Part2 % docker-compose exec kafka kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic final \
  --from-beginning
{"task_id": "cf607c61", "from_agent": "reviewer", "status": "approved", "answer": "The CAP theorem is a fundamental concept in distributed systems that states it is impossible for a distributed data store t
o simultaneously guarantee all three of the following: consistency, availability, and partition tolerance. Consistency refers to ensuring that data is accurate and up-to-date across all nodes in the system,
 availability refers to ensuring that the system is always accessible and responsive, and partition tolerance refers to the ability of the system to continue functioning even in the presence of network part
itions or failures. This trade-off is crucial because achieving consistency and availability simultaneously can lead to significant performance and latency penalties, while prioritizing partition tolerance
can compromise data consistency and availability. As a result, the CAP theorem matters because it provides a framework for understanding the limitations and design choices of distributed systems, and inform
s the development of systems that balance these competing requirements.", "feedback": "Clear explanation of the CAP theorem and its implications."}
```

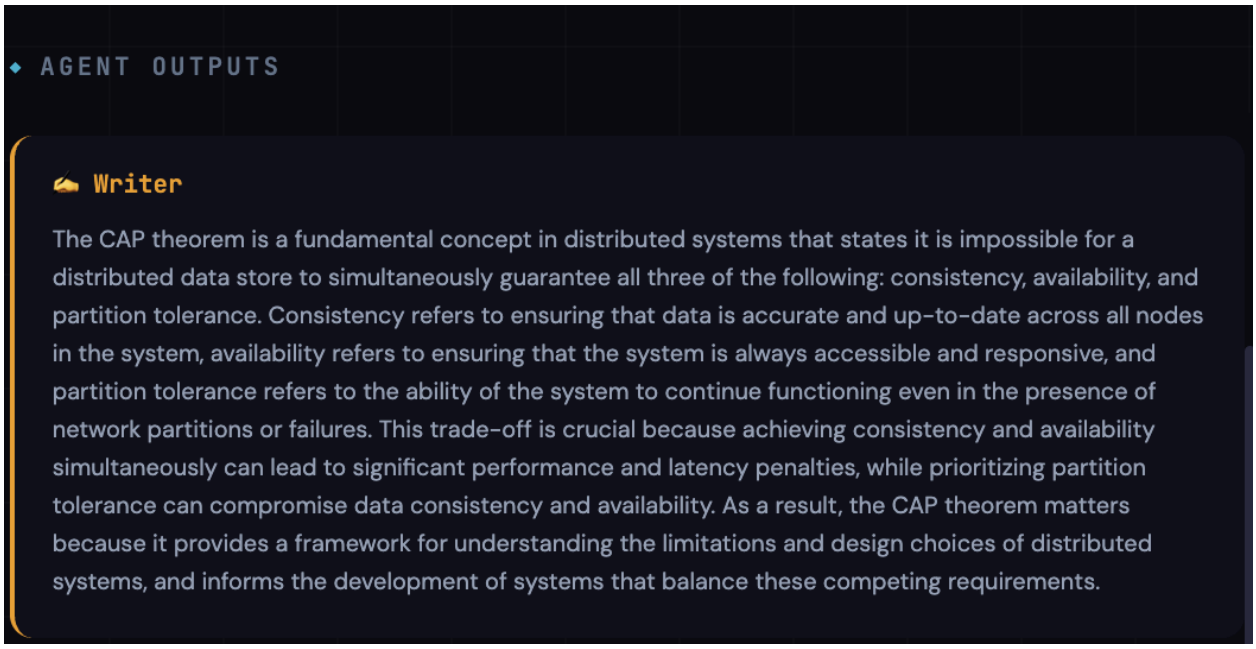
{"task_id": "cf607c61", "from_agent": "reviewer", "status": "approved", "answer": "The CAP theorem is a fundamental concept in distributed systems that states it is impossible for a distributed data store to simultaneously guarantee all three of the following: consistency, availability, and partition tolerance. Consistency refers to ensuring that data is accurate and up-to-date across all nodes in the system, availability refers to ensuring that the

system is always accessible and responsive, and partition tolerance refers to the ability of the system to continue functioning even in the presence of network partitions or failures. This trade-off is crucial because achieving consistency and availability simultaneously can lead to significant performance and latency penalties, while prioritizing partition tolerance can compromise data consistency and availability. As a result, the CAP theorem matters because it provides a framework for understanding the limitations and design choices of distributed systems, and informs the development of systems that balance these competing requirements.", "feedback": "Clear explanation of the CAP theorem and its implications."}

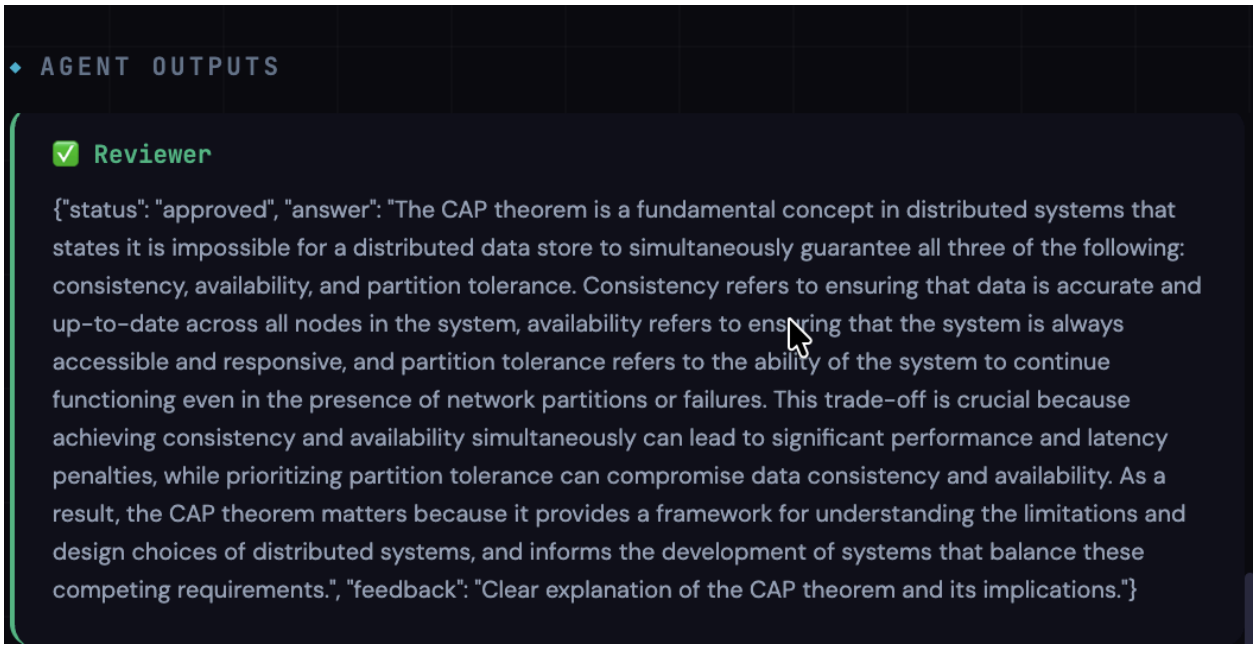
Planner:



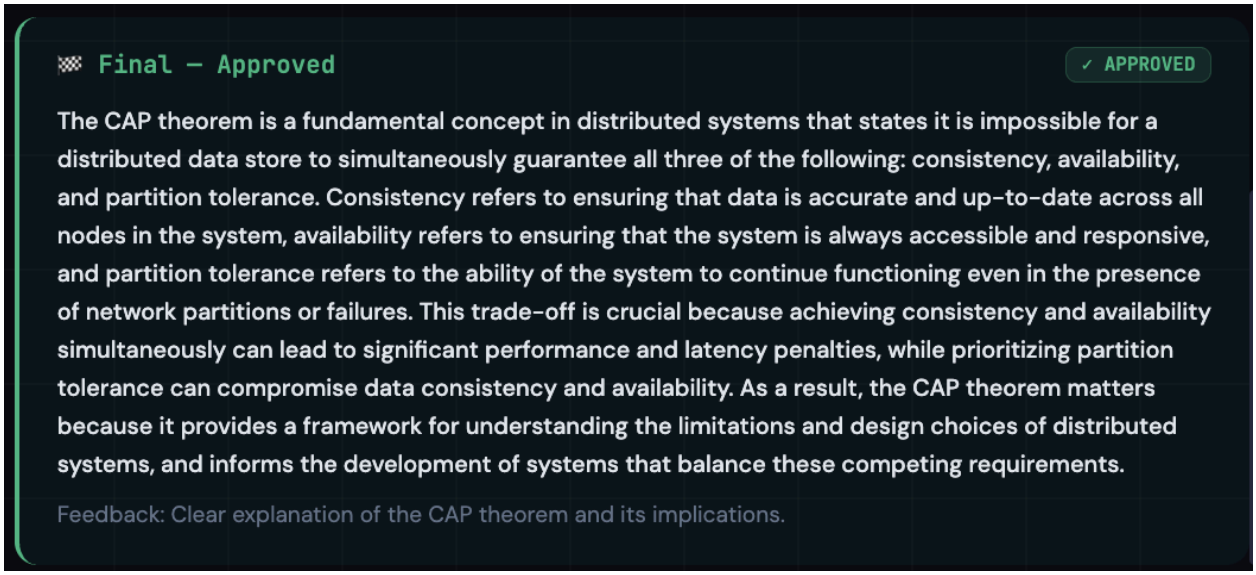
Writer:



Reviewer:



Final:



- A short report explaining how Kafka helped your agents coordinate.

In this system, three independent agents Planner, Writer, and Reviewer collaborate to answer a question without ever calling each other directly. Apache Kafka serves as the communication backbone that makes this possible.

Each agent is decoupled: the Planner only knows about the inbox and tasks topics. The Writer only knows about tasks and drafts. The Reviewer only knows about drafts and final. No agent holds a reference to another agent. This means each agent can be started, stopped, or restarted independently without breaking the pipeline.

Kafka topics act as persistent message queues between agents. When the Planner finishes generating a plan, it publishes a message to the tasks topic and its job is done. The Writer consumes from tasks at its own pace, generates an answer, and publishes to drafts. The Reviewer then picks up from drafts and publishes the final approval to final. This producer-consumer model ensures no message is lost even if an agent is temporarily unavailable, Kafka holds the message until the consumer is ready.

Kafka also provides natural flow control. Each agent processes one message at a time and only moves to the next after completing its work, preventing any agent from being overwhelmed.

In summary, Kafka transformed what would have been tightly coupled function calls into a loosely coupled, asynchronous pipeline where agents coordinate purely through message passing, making the system scalable, resilient, and easy to extend with additional agents.